Task 1: Decision Tree Classifier

Name: Manish Pawar
Internship: Soft Nexis Technology - Data Science & ML in Python
Dataset: Iris Flower Dataset

Objective

The objective of this task is to build a Decision Tree Classifier using Scikit-Learn, train it on the Iris dataset, test it on unseen data, evaluate the model using accuracy, classification report and confusion matrix, and study overfitting using tree depth experiments.

Output Summary

Dataset shape: (150, 5)
Species: ['setosa' 'versicolor' 'virginica']

First five rows:

	sepal length (cm)	sepal width (cm)	...	petal width (cm)	species
0	5.1	3.5	...	0.2	setosa
1	4.9	3.0	...	0.2	setosa
2	4.7	3.2	...	0.2	setosa
3	4.6	3.1	...	0.2	setosa
4	5.0	3.6	...	0.2	setosa

[5 rows x 5 columns]

Training samples: 120
Testing samples: 30
Training class distribution: {np.int64(0): np.int64(40), np.int64(1): np.int64(40), np.int64(2): np.int64(40)}

Model training complete!
Number of leaves in tree: 5
Tree depth: 3

Actual labels: [0 2 1 1 0 1 0 0 2 1]
Predicted labels: [0 2 1 1 0 1 0 0 2 1]

New flower measurements: [5.1, 3.5, 1.4, 0.2]
Predicted species: setosa
Confidence: [[1. 0. 0.]]

Accuracy: 96.67%

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	1.00	0.90	0.95	10
virginica	0.91	1.00	0.95	10
accuracy			0.97	30
macro avg	0.97	0.97	0.97	30
weighted avg	0.97	0.97	0.97	30

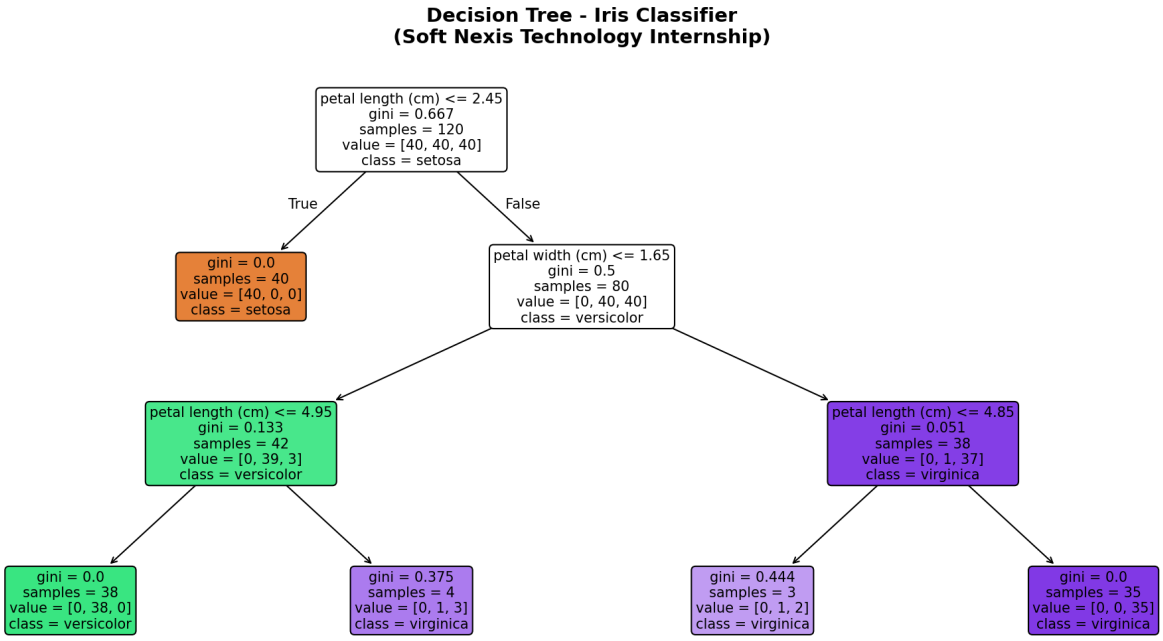
Confusion Matrix:
[[10 0 0]
[0 9 1]
[0 0 10]]

Decision tree visualization saved as decision_tree_visual.png
Depth vs Accuracy Results:

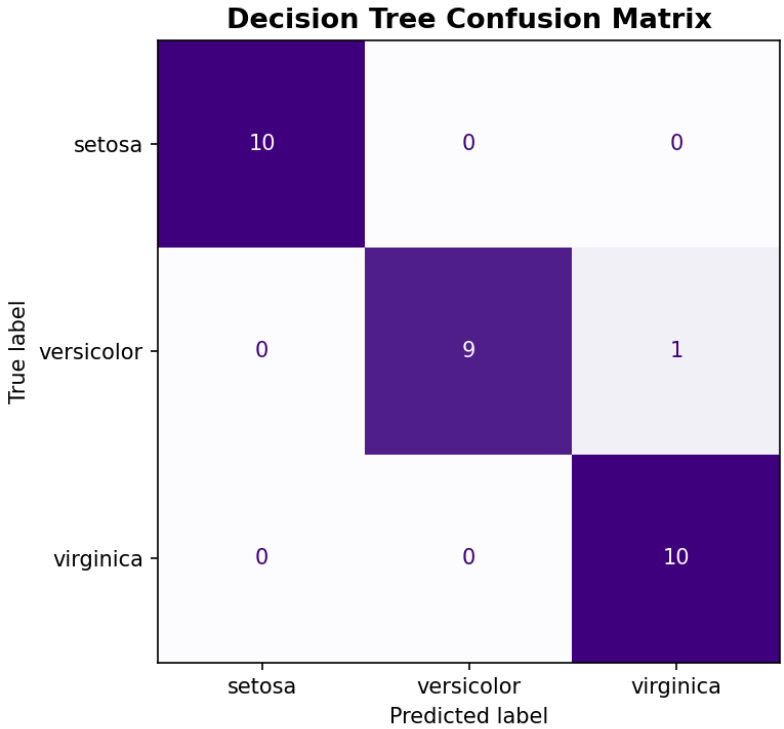
	max_depth	training_accuracy	test_accuracy
0	1	0.666667	0.666667
1	2	0.966667	0.933333
2	3	0.983333	0.966667
3	4	0.991667	0.933333
4	5	1.000000	0.933333
5	10	1.000000	0.933333
6	None	1.000000	0.933333

Depth vs accuracy graph saved as depth_vs_accuracy.png

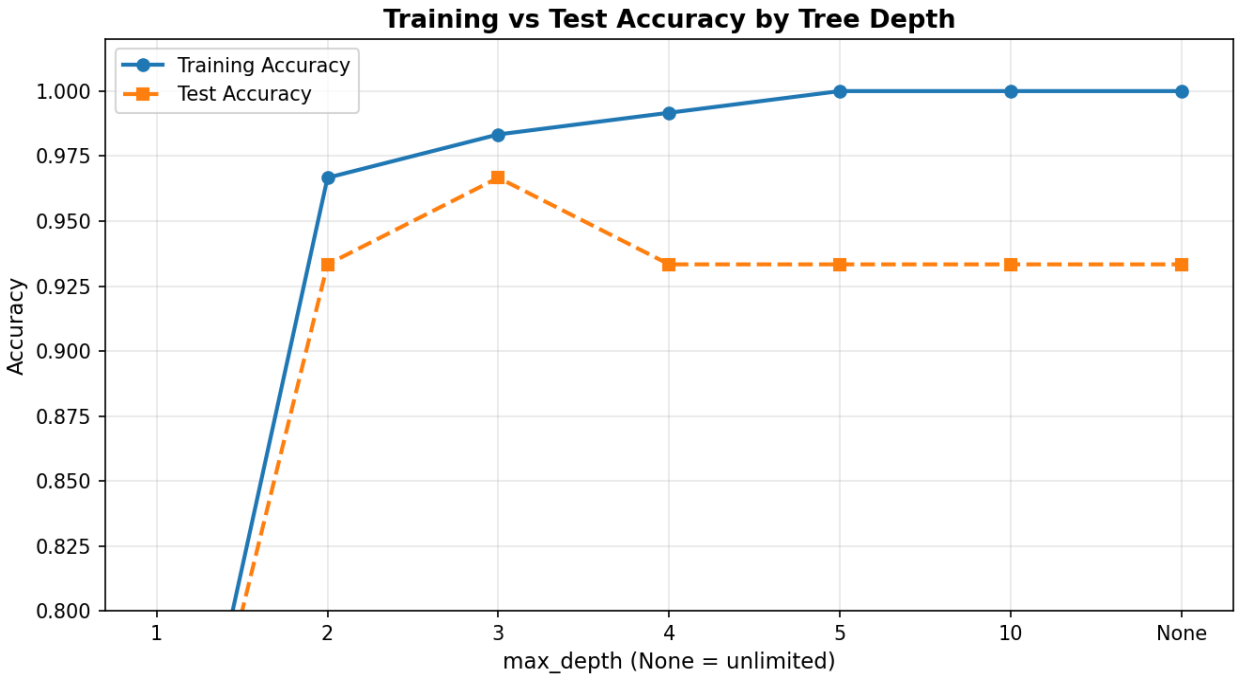
Decision Tree Visualization



Confusion Matrix



Depth vs Accuracy Graph



250-350 Word Report: Overfitting and Random Forest

Overfitting means a machine learning model learns the training data too closely instead of learning the general pattern. In simple words, the model memorises the examples it has already seen. Because of this, it can give very high accuracy on training data, but it may perform poorly on new unseen data. A Decision Tree can easily overfit because it keeps splitting the data into smaller and smaller parts. If the tree becomes too deep, it may create rules for very specific samples instead of useful rules for the whole dataset.

In this task, the Decision Tree was trained on the Iris dataset. The tree used flower measurements such as sepal length, sepal width, petal length and petal width to predict the species of flower. The model was tested using separate test data that was not used during training. The confusion matrix and accuracy score show how well the model classified unseen flowers. The depth vs accuracy graph is important because it shows the relation between tree complexity and performance. When tree depth increases, training accuracy usually becomes higher. However, if test accuracy does not improve or starts decreasing, it means the tree is overfitting.

Random Forest solves overfitting by using many Decision Trees instead of only one tree. Each tree is trained on a random part of the data and also considers random features while making splits. After that, all trees vote for the final answer. Because the final prediction is based on many trees, the mistakes of one tree are balanced by other trees. This makes Random Forest more stable and reliable than a single Decision Tree. It reduces overfitting and gives better performance on new data.

Conclusion

The Decision Tree model successfully classified Iris flower species and produced all required outputs. The experiment also showed why controlling tree depth is important and how Random Forest improves generalization by combining multiple trees.

Complete Python Code

```
# Task 1: Decision Tree Classifier on Iris Dataset
# Name: Manish Pawar
# Internship: Soft Nexis Technology - Data Science & ML in Python

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, ConfusionMatrixDisplay

# 1. Load and prepare data
iris = load_iris()
X = iris.data
y = iris.target

df = pd.DataFrame(X, columns=iris.feature_names)
df['species'] = iris.target_names[y]
print('Dataset shape:', df.shape)
print('Species:', df['species'].unique())
print('\nFirst five rows:')
print(df.head())

# 2. Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
print('\nTraining samples:', X_train.shape[0])
print('Testing samples:', X_test.shape[0])
print('Training class distribution:', dict(zip(*np.unique(y_train, return_counts=True))))

# 3. Train Decision Tree
# max_depth=3 helps control model complexity and reduces overfitting.
dt_model = DecisionTreeClassifier(max_depth=3, random_state=42)
dt_model.fit(X_train, y_train)
print('\nModel training complete!')
print('Number of leaves in tree:', dt_model.get_n_leaves())
print('Tree depth:', dt_model.get_depth())

# 4. Make predictions
y_pred = dt_model.predict(X_test)
print('\nActual labels:   ', y_test[:10])
print('Predicted labels:', y_pred[:10])

new_flower = [[5.1, 3.5, 1.4, 0.2]]
prediction = dt_model.predict(new_flower)
prediction_proba = dt_model.predict_proba(new_flower)
print('\nNew flower measurements:', new_flower[0])
print('Predicted species:', iris.target_names[prediction[0]])
print('Confidence:', prediction_proba)

# 5. Evaluate model
accuracy = accuracy_score(y_test, y_pred)
print(f'\nAccuracy: {accuracy*100:.2f}%')
print('\nClassification Report:')
print(classification_report(y_test, y_pred, target_names=iris.target_names))

cm = confusion_matrix(y_test, y_pred)
print('Confusion Matrix:')
print(cm)

# Save confusion matrix image
fig, ax = plt.subplots(figsize=(7, 5))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=iris.target_names)
disp.plot(ax=ax, cmap='Purples', values_format='d', colorbar=False)
```

```
ax.set_title('Decision Tree Confusion Matrix', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig('confusion_matrix_dt.png', dpi=150)
plt.close()

# 6. Save decision tree visualization
fig, ax = plt.subplots(figsize=(14, 7))
plot_tree(
    dt_model,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    filled=True,
    rounded=True,
    fontsize=10,
    ax=ax
)
ax.set_title('Decision Tree - Iris Classifier\n(Soft Nexis Technology Internship)', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('decision_tree_visual.png', dpi=150, bbox_inches='tight')
plt.close()
print('\nDecision tree visualization saved as decision_tree_visual.png')

# 7. Experiment with tree depth to study overfitting
depths = [1, 2, 3, 4, 5, 10, None]
train_acc = []
test_acc = []

for depth in depths:
    model = DecisionTreeClassifier(max_depth=depth, random_state=42)
    model.fit(X_train, y_train)
    train_acc.append(model.score(X_train, y_train))
    test_acc.append(model.score(X_test, y_test))

results = pd.DataFrame({
    'max_depth': [str(d) for d in depths],
    'training_accuracy': train_acc,
    'test_accuracy': test_acc
})
print('\nDepth vs Accuracy Results:')
print(results)

fig, ax = plt.subplots(figsize=(9, 5))
depth_labels = [str(d) for d in depths]
ax.plot(depth_labels, train_acc, 'o-', label='Training Accuracy', linewidth=2)
ax.plot(depth_labels, test_acc, 's--', label='Test Accuracy', linewidth=2)
ax.set_title('Training vs Test Accuracy by Tree Depth', fontsize=13, fontweight='bold')
ax.set_xlabel('max_depth (None = unlimited)', fontsize=11)
ax.set_ylabel('Accuracy', fontsize=11)
ax.legend(fontsize=10)
ax.set_ylim(0.8, 1.02)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('depth_vs_accuracy.png', dpi=150)
plt.close()
print('Depth vs accuracy graph saved as depth_vs_accuracy.png')
```